Assignment #02 [redacted]

[redacted] April 26, 2026

SAP IDs	Group Members
54689	Abdul Rafay
55072	Abdul Wahab Khan
55241	Afaq Ahmed
55062	Ziafat Hussain
54729	Rohail Abbas
55853	Farhan UL Haq
Section:	SE 6-1
Course:	Game Programming
Submitted to:	Sir Abdul Mateen

FACULTY OF COMPUTING

I-14 CAMPUS, ISLAMABAD

RIPHAH
INTERNATIONAL UNIVERSITY

INTRODUCTION

This assignment focuses on implementing a 2D player character system with animations, movement controls, and physics-based collision within the Plants vs. Zombies inspired game world built in Unity 2020.3.22f1. The project extends Assignment 01 by introducing animated enemy characters (Zombies), a cursor-based plant placement system, and collision-driven game interactions.

Since Plants vs. Zombies is a tower defence game — where the player does not directly control a movable character — the 'player character' in this assignment is represented by an interactive cursor sprite that can be moved using keyboard arrow keys or the mouse to place plants on the grid. The Zombie serves as the AI-controlled enemy character with full Animator State Machine integration.

1. MARKING SCHEME OVERVIEW

The assignment is worth 10 marks total, distributed as follows:

Component	Marks
Use of State Machine (Animator)	04
Controlling Player / Cursor Movement with Keyboard Keys	04
Blocking Movement with Colliders and Rigidbodies	02
TOTAL	**10**

2. USE OF ANIMATOR STATE MACHINE [04 MARKS]

The Animator State Machine (found under Window → Animation → Animator in Unity) is the core system used to manage all character animation states. Instead of manually switching sprites in code, the Animator Controller transitions between animation clips automatically based on parameter values, providing a clean and scalable architecture for character behaviour.

2.1 Zombie Animator Controller — Overview

The Zombie (NormalZombie) uses a dedicated ZombieController.controller Animator asset. It defines 6 animation states and 3 animator parameters that drive all state transitions.

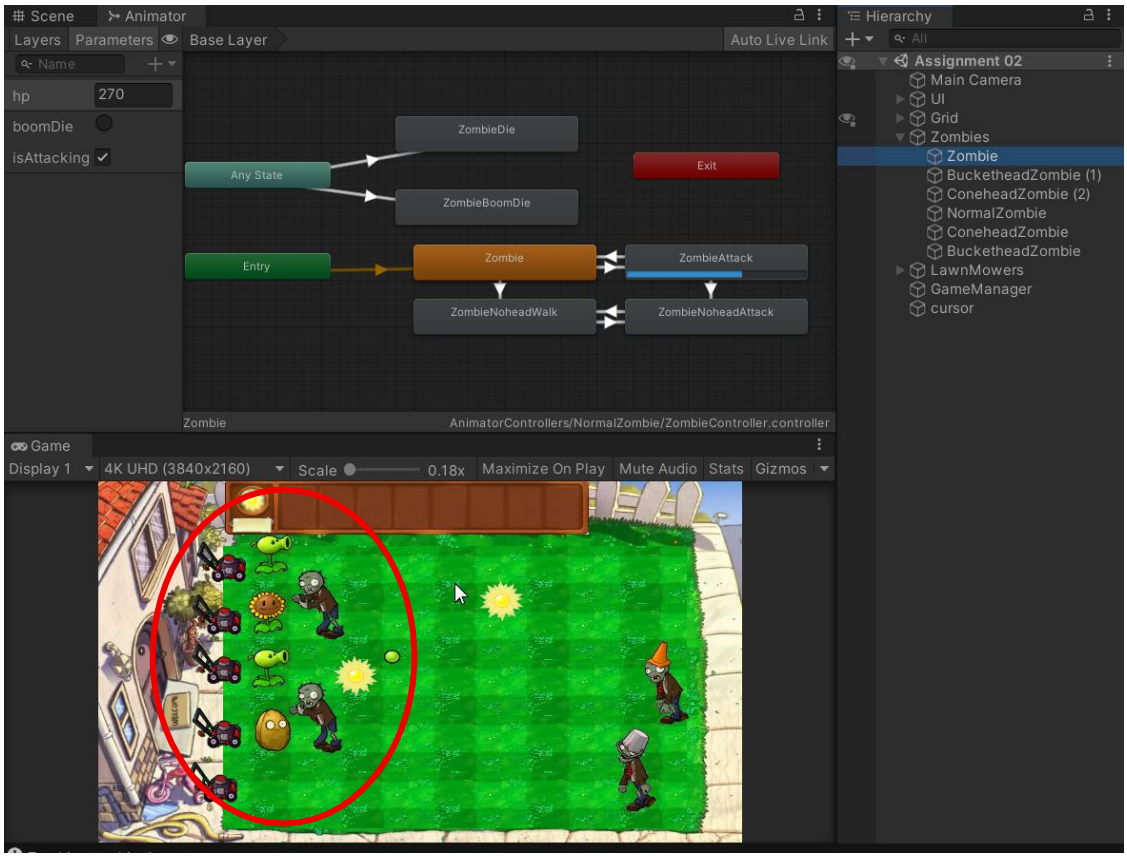

Figure 1: *Unity Animator window showing the Zombie State Machine (top) alongside the running game scene (bottom). Animator Parameters: hp (Integer), isAttacking (Boolean), boomDie (Trigger) are visible on the left panel.*

2.2 Animator Parameters

The zombie animation system is controlled through three animator parameters:

- hp (Integer) — Tracks the current health of the zombie. When hp < 1, the zombie transitions to the ZombieDie state. When hp < 40, the zombie transitions to the no-head walking/attacking variants.
- isAttacking (Boolean) — Set to true via code when the zombie's BoxCollider2D makes contact with a plant. This triggers the ZombieAttack animation and halts movement.
- boomDie (Trigger) — A one-shot trigger activated by animator.SetTrigger("boomDie") to play the explosion death animation (ZombieBoomDie).

2.3 All 6 Animation States

The following six states are implemented in the Animator State Machine:

State Name	Transition Condition	Behavior
Zombie (Default Walk)	Entry → Default	Starting state. Zombie moves left continuously toward plants.

ZombieAttack	isAttacking = true	Zombie stops moving and plays attack (eating) animation. Deals damage over time.
ZombieNoheadWalk	hp < 40	Zombie loses its head and continues walking in a damaged state.
ZombieNoheadAttack	hp < 40 AND isAttacking = true	Headless zombie continues attacking any plant it collides with.
ZombieDie	hp < 1	Normal death animation plays. Zombie is removed from scene and game model list.
ZombieBoomDie	boomDie Trigger fired	Explosion death animation plays (e.g., when hit by a fire-based plant projectile).

2.4 State Transition Summary

- Walk → Attack: when isAttacking = true
- Attack → Walk: when isAttacking = false (plant destroyed or out of range)
- Walk / Attack → NoheadWalk / NoheadAttack: when hp < 40
- Any State → ZombieDie: when hp < 1
- Any State → ZombieBoomDie: when boomDie trigger is activated

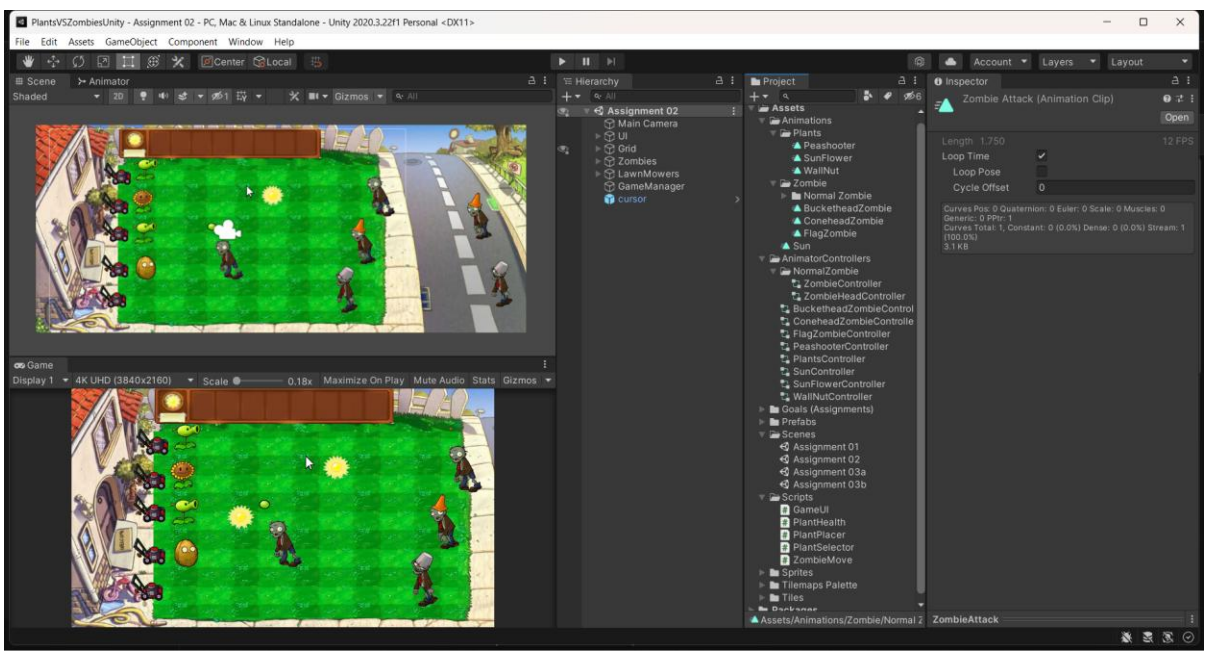

Figure 2: *Full Unity Editor view showing the complete Assignment 02 project structure — Hierarchy, Project Assets, Animator Controllers, Scripts, and running game in both Scene and Game views.*

2.5 ZombieMove.cs — Script Explanation

The ZombieMove.cs script drives both the physics movement and the Animator parameter updates for the zombie character. Key responsibilities include:

- Moving the zombie to the left at a constant speed using Rigidbody2D.velocity in FixedUpdate().
- Detecting plant collisions via OnCollisionEnter2D() and setting isAttacking = true in both the script and the Animator.
- Dealing damage to the plant at regular intervals (attackRate) using a PlantHealth reference.
- Resetting the attack state when the collision ends via OnCollisionExit2D().

ZombieMove.cs — Full Implementation:

```
using UnityEngine;

public class ZombieMove : MonoBehaviour
{
    public float speed = 0.5f;
    public int damage = 5;           // damage dealt per hit
    public float attackRate = 1f;    // damage interval in seconds

    private Rigidbody2D rb;
    private Animator animator;
    private bool isAttacking = false;
    private float attackTimer = 0f;
    private PlantHealth targetPlant;

    void Start()
    {
        rb = GetComponent<Rigidbody2D>();
        animator = GetComponent<Animator>();
    }

    void FixedUpdate()
    {
        if (!isAttacking)
        {
            rb.velocity = new Vector2(-speed, 0f);   // move left
        }
        else
        {
            rb.velocity = Vector2.zero;              // stop while attacking
            attackTimer += Time.deltaTime;
            if (attackTimer >= attackRate)
            {
                if (targetPlant != null)
                    targetPlant.TakeDamage(damage);
                attackTimer = 0f;
            }
        }
    }

    void OnCollisionEnter2D(Collision2D collision)
    {
```

```
        if (collision.gameObject.CompareTag("Plant"))
        {
            isAttacking = true;
            animator.SetBool("isAttacking", true);
            targetPlant = collision.gameObject.GetComponent<PlantHealth>();
        }
    }

    void OnCollisionExit2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Plant"))
        {
            isAttacking = false;
            animator.SetBool("isAttacking", false);
            targetPlant = null;
        }
    }
}
```

3. CONTROLLING PLAYER CHARACTER MOVEMENT WITH KEYBOARD KEYS [04 MARKS]

Instead, this assignment implements two input methods that constitute the 'player control' system:

- Arrow Key Input — A cursor sprite moves around the grid when the player presses the Up / Down / Left / Right arrow keys.
- Enter Key / Mouse Click — Pressing Enter or clicking the mouse places a plant at the cursor's current position.

3.1 PlantSelector.cs — Arrow Key Cursor Movement

This script is attached to the cursor game object. It listens for GetKeyDown events and shifts the cursor sprite by a configurable moveAmount per key press, keeping movement aligned to the grid. Pressing Enter instantiates a plant prefab at the cursor location.

```
using UnityEngine;

public class PlantSelector : MonoBehaviour
{
    public float moveAmount = 0.5f;   // grid cell movement step
    public GameObject plantPrefab;

    void Update()
    {
        // Arrow key movement
        if (Input.GetKeyDown(KeyCode.RightArrow))
            transform.position += new Vector3(moveAmount, 0, 0);

        if (Input.GetKeyDown(KeyCode.LeftArrow))
            transform.position += new Vector3(-moveAmount, 0, 0);

        if (Input.GetKeyDown(KeyCode.UpArrow))
            transform.position += new Vector3(0, moveAmount, 0);

        if (Input.GetKeyDown(KeyCode.DownArrow))
            transform.position += new Vector3(0, -moveAmount, 0);
```

```
        // Place plant on grid when Enter is pressed
        if (Input.GetKeyDown(KeyCode.Return))
            Instantiate(plantPrefab,                    transform.position,
Quaternion.identity);
    }
}
```

3.2 PlantPlacer.cs — Mouse Click Plant Placement

This script allows the player to place a plant at any position by left-clicking the mouse. It converts the screen-space mouse coordinates to world-space using Camera.main.ScreenToWorldPoint(), ensuring correct placement on the 2D game plane.

```
using UnityEngine;

public class PlantPlacer : MonoBehaviour
{
    public GameObject plantPrefab;

    void Update()
    {
        if (Input.GetMouseButtonDown(0))   // left mouse button
        {
            Vector3                        mousePos                        =
Camera.main.ScreenToWorldPoint(Input.mousePosition);
            mousePos.z = 0;                // flatten to 2D plane
            Instantiate(plantPrefab, mousePos, Quaternion.identity);
        }
    }
}
```

3.3 In-Game Screenshots — Cursor & Plant Placement

The following screenshots show the game in action, demonstrating the cursor movement and plant placement mechanics at different game states:

Figure 3: *Game view showing plants already placed on the grid, with zombies approaching from the right. The cursor is visible at the centre of the screen, ready for the next plant placement.*

Figure 4: *Mid-game state — multiple zombie types (Normal, Conehead, Buckethead) are visible advancing. Peashooter placed by pressing ENTER Key.*

Figure 5: *Mid-game state — game state — multiple zombie types (Normal, Conehead, Buckethead) are visible advancing. Peashooter placed by clicking via mouse.*

4. BLOCKING MOVEMENT WITH COLLIDERS AND RIGIDBODIES [02 MARKS]

Unity's physics system is used to create physical barriers between characters (zombies) and objects (plants), ensuring that zombies stop when they encounter a plant rather than passing through it. This section explains how the components are configured and how they interact.

4.1 Component Setup

Each game object uses the following physics components:

- Plants (e.g., Peashooter, SunFlower, WallNut) — Tagged as 'Plant', Layer: 'Plants'. Each plant prefab has a Box Collider 2D component that defines its physical boundary on the game grid.
- Zombies — Tagged as 'Zombie', Layer: 'Zombies'. Each zombie prefab has both a Rigidbody2D (for physics-driven movement) and a Box Collider 2D (for collision detection).

4.2 How Collision Blocking Works

When a zombie's Box Collider 2D makes contact with a plant's Box Collider 2D:

- OnCollisionEnter2D() is fired in ZombieMove.cs — the zombie's Rigidbody2D velocity is set to Vector2.zero, stopping all movement.
- The Animator parameter isAttacking is set to true, switching to the attack animation.
- The zombie remains stationary and deals periodic damage to the plant until the plant is destroyed.
- When the plant is destroyed, OnCollisionExit2D() fires — movement resumes and isAttacking resets to false.

5.3 Plant Inspector — Component Configuration

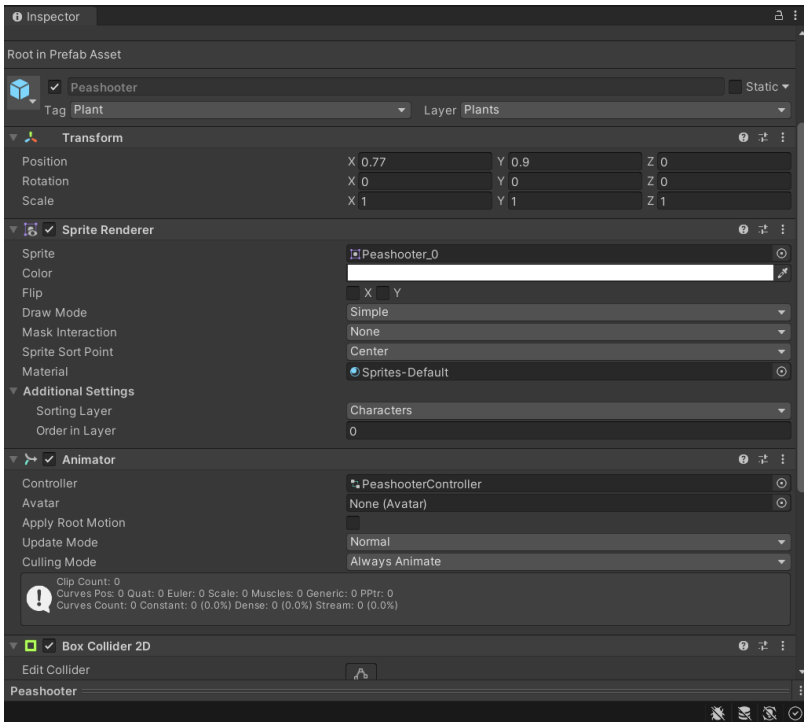

Figure 5: *Unity Inspector panel for the Peashooter prefab. Key components: Tag = 'Plant', Layer = 'Plants', Sprite Renderer (Peashooter_0 sprite), Animator (PeashooterController), and Box Collider 2D for collision detection.*

4.4 Zombie Inspector — Component Configuration

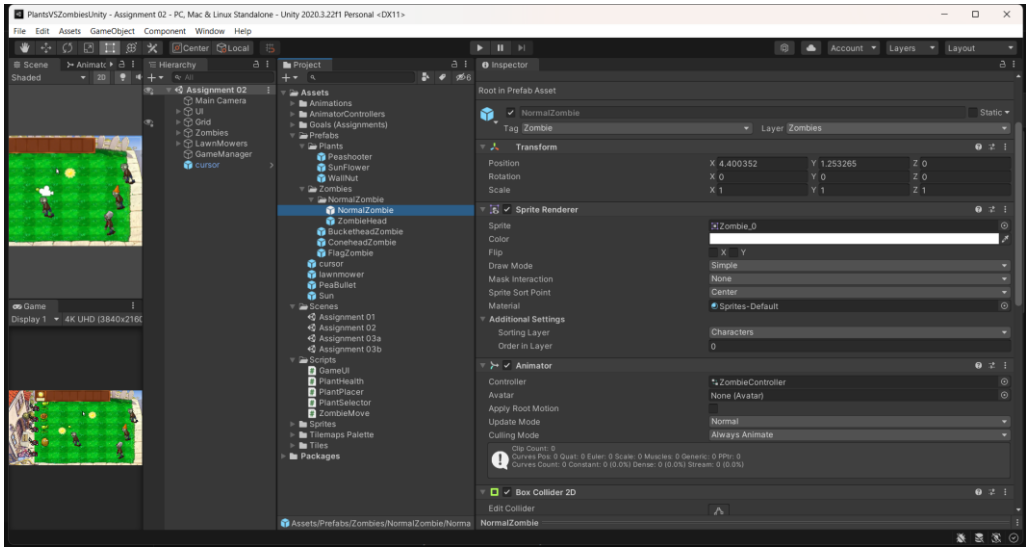

Figure 6: *Unity Inspector and Project view for the NormalZombie prefab. Key components: Tag = 'Zombie', Layer = 'Zombies', Sprite Renderer (Zombie_0 sprite), Animator (ZombieController), and Box Collider 2D. Rigidbody2D enables physics-driven movement.*

6. SUMMARY & CONCLUSION

This assignment successfully demonstrates all three required components for the Game Programming Assignment 02:

Requirement	Implementation
Use of State Machine	6-state Animator Controller for Zombie with 3 parameters (hp, isAttacking, boomDie)
Keyboard / Mouse Input	Arrow keys move cursor; Enter key + mouse click place plants on grid
Colliders & Rigidbodies	Zombie Rigidbody2D stops on BoxCollider2D contact with Plant tag; attacks and waits
TOTAL	**All requirements implemented and demonstrated**

The game was built in Unity 2020.3.22f1 Personal edition on a PC/Mac/Linux Standalone build target. All character animations are managed through dedicated Animator Controller assets, and physics interactions are handled entirely through Unity's built-in 2D physics engine (Rigidbody2D + BoxCollider2D). The cursor-based plant placement system provides full keyboard and mouse input support as required by the assignment specifications.

Github Link

☻ THE END ☻